

Elvex – Documentation

Lionel Clément

Version 2.14.0

Introduction

Elvex is a surface realisation system based on typed feature structures, lexical resources and syntagmatic rules. It takes as input a feature structure representing a concept or a generation request, combines it with a lexicon and a grammar, and produces one or more textual realizations.

The central idea of **Elvex** is to describe generation by means of rules whose right-hand side symbols exchange information through inherited and synthesized feature structures. Inherited structures, written $\downarrow_i$, transmit information from a rule to its components, while synthesized structures, written $\Downarrow_i$, return information produced by these components. This makes it possible to express agreement, selection, optional constituents, alternatives and lexical constraints in a uniform way.

This document introduces the main elements of the language: feature structures, lexicons, rules, operational semantics and command-line tools. It also describes mechanisms used during generation, such as optional symbols, alternatives, deferred evaluation, tracing and linearization constraints. The examples are intended to show both the concrete syntax and the style of grammar writing expected by **Elvex**.

The documentation is organized progressively. The first sections present the linguistic and formal notions used by the system. The following sections describe the syntax of grammars and lexicons, then the operational statements available inside rules. The final sections explain how to run **Elvex**, how to debug grammars, and how to build or consult compacted lexicons.

1 Full Example: Getting Started

An example will familiarize the reader with **Elvex** before we go any further. Let us take the case of the text generation of a nominal phrase in French.

Example

```
1 NP → det N {
2   ↓2 = ↑;
3   ↓1 = ↓2;
4   ↑ = ↓2;
5 }
6
7 N → adj N {
8   [mod:<$Head::$Tail>, $Rest];
9   [number:$inhNb] ⊂ ↑;
10  ↓1 = [number:$inhNb, gender:$synthGd, $Head];
11  ↓2 = [mod:$Tail, $Rest];
12  [gender:$synthGd] ⊂ ↓2;
13  ↑ = ↓2 ∪ [qual:yes];
14 }
15
16 N → n {
17   [mod:NIL];
18   ↓1 = ↑;
19   ↑ = ↓1;
20 }
```

1.1 Syntagmatic description (*Context-Free Grammar rules*)

The first rule of grammar (line 1) defines **NP** as consisting of **det** and **N** from left to right.

det is a word; it is defined in the lexicon that we will see later.

Lines 7) and 16) follow the same logic and we have the context-free grammar:

```
NP → det N
N → adj N
N → n
```

The only sequence we can generate with this simple example is a noun, preceded by a determiner and possibly one or several adjectives.

Syntagmatic rules are therefore made to describe the order of components of phrases, sentences and speeches.

1.2 Operational semantics

Each rule contains a set of operations. These operations describe the functional properties of the language, which may or may not depend on the order of the words.

The basics

We will explain this semantics through the example given to the beginning of this section before giving details.

- The lines 2) and 3) express the nature of the links that exist between NP, the name N and its determiner **det**; the main one being that agreement in gender and number occurs between nouns and their specifier.
 - Line 2) assigns the given nominal phrase properties to the noun. The attribute of the name is noted $\downarrow 2$ (this is the second term of the right part of the rule), and nominal phrase's is noted $\uparrow$.
 - Line 3) retrieves information from analyses of N to assign them to the determiner. The attribute of the determiner is noted $\downarrow 1$ (this is the first term of the right part of the rule), the attribute from the name N is noted $\downarrow 2$.
 - Line 4) assigns the analyse of N to the synthesized attribute NP.

With the two first lines, the noun will be chosen in the lexicon according to the concept to be expressed (attribute of the nominal phrase $\uparrow$). A female singular determiner will be chosen if the gender of the noun is feminine and the nominal phrase is singular.

This mechanism, which makes it possible to articulate lexical choices according to the concepts and syntactic properties according to lexical properties is at the heart of **Elvex** and implements two types of attributes: inherited attributes (noted $\uparrow$, $\downarrow_i$) and synthesized attributes (noted $\uparrow$, $\downarrow_i$).

Inherited attributes are calculated from left to right of the syntagmatic rules. Synthesized attributes are calculated from right to left. To put it simply, we can say that the inherited attributes convey information from the concept to the lexicon, and that the synthesized attributes convey information from the lexicon to higher layers of the syntactic analysis. Thus inherited attributes allow you to propagate the input

to the lexicon to make a lexical choice, whereas conversely, synthesized attributes allow to retrieve the lexical information of the chosen name according to the concept expressed and to propagate this information (grammatical features, selection restrictions, phraseological units constraints, etc.) to the entire nominal phrase.

- The lines 8) to 13) allow you to set the gender and number agreement of the noun and adjective and also to propagate the concepts that produce adjectives. We notice that in French, the number is a semantism expressed directly as a sign associated with the noun, whereas gender is a grammatical feature given by the lexicon. The first will be propagated from the concept to the noun, the second will be propagated from the lexicon to the layers regulating the agreement.
 - Line 8) is a *guard*, *i. e.* a condition to realize $N \rightarrow \text{adj } N$. It expresses that the realization of an adjective is conditioned by the existence of a list $\langle \$\text{Head}::\$ \text{Tail} \rangle$ as the value of an attribute **mod** in the inherited attribute $\uparrow$.
 $\$ \text{Head}$ and $\$ \text{Tail}$ are two variables that respectively define the head and the tail of a list, *i. e.* respectively the first item in the list and the rest of the list.
 if the condition is verified, $\$ \text{Head}$, $\$ \text{Tail}$ and $\$ \text{Rest}$ will be affected by values given by the subsumption¹ values of $[\text{mod}:\langle \$ \text{Head}::\$ \text{Tail} \rangle, \$ \text{Rest}]$ by $\uparrow$.
 - Line 9) assign to the variable $\$ \text{inhNb}$ (*Inherited noun*) the value of the **number** feature of the inherited attribute $\uparrow$. This allows you to retrieve the number given to the nominal phrase.
 - Line 10) assigns to the inherited attribute of the adjective the head of the list $\$ \text{Head}$ unified² with a structure that gives the synthesized gender $\$ \text{synthGd}$ (*synthesized gender*) and the inherited number $\$ \text{inhNb}$.
 - Line 11) assigns to the inherited attribute of the noun all values passed by inheritance ($\$ \text{Rest}$) and adds the feature **mod** with the value $\$ \text{Tail}$, which is the tail of the list that brings together all other modifiers of the noun (other adjective or a proposal for example).

¹A feature structure **A** subsumes a feature structure **B** iff all the features of **B** are present in **A** with the same values or with values that subsume those of the values of **A**.

²The unification of a feature structure **A** with a feature structure **B**, is the smallest feature structure **C** such that **A** subsumes **C** and **B** subsumes **C**.

- Line 12) allows you to assign the variable `$synthGd` which is the gender synthesized from the noun.
- Finally line 13) assigns to the synthesized attribute of the phrase the synthesized attribute of the noun and adds a feature `qual` which marks that this noun phrase is qualified (which makes it possible to restrict others modifications).
- The rules on lines 17) to 19) are not specific, except for the custody of rule 17) which implements the value `NIL`.
 - Line 17) is a guard which allows us to prevent the inherited attribute `↑` to contain any feature `mod`. `mod:NIL` means that the attribute `mod` does not exist in `↑`.
 - Lines 18) and 19) do not require any further comment.

It becomes clear by examining this simple example that the order of operational rules is not the one given by grammar, but the one dictated by the availability of operands. Here, a possible order of application of the rules is 8), 9), 11), 12), 10), and 13).

In the current version of *Elvex*, this order is calculated from the availability of operands during rule evaluation.

The advice I can give to develop a grammar is to write the rules as the synthesized attributes are available.

We see that the operational statements of syntagmatic rules are not evaluated strictly from left to right or from top to bottom, but according to the availability of the attributes they depend on.

Before going any further, let us give definitions on the ratings that allow us to understand the syntax of the rules.

Empty rules, optional terms

It is possible to write empty rules, *i.e.* components that have no realization in text production. It is also possible to put optional elements written between parentheses `()`.

Example: The following rule gives all possible combinations for clitic pronouns placed before the verb.

`CLITICS → ( clr ) ( cld ) ( cla ) ( cld ) ( clg ) ( cll )`

In French, the dative clitic (`cld`) is sometimes placed before, sometimes placed after the accusative clitic (`cla`) according to the person of the pronoun.

- (1) a. *Jean me le donne*
 Jean **to_me**(dative) **it**(accusative) gives
 Jean gives it to me
- b. *Jean le lui donne*
 Jean **it**(accusative) **to_him**(dative) gives
 Jean gives it to him.

According to this rule, the phrase CLITICS can also be empty, due to the fact that each term is optional.

Paradigms

Each element of a syntagmatic rule can be a list of terms separated by the character | instead of being a single term. In this case, the product phrase is constructed with one of the terms in the list.

Example: The following rule allows you to build a sentence without a subject, with a nominative clitic subject **cln** (*je, tu, il, ...*), or with a subject as a nominal group.

$S \rightarrow (\text{cln} \mid \text{NP}) \text{ VP}$

A grammar written by using massively rules with alternatives and optional terms is equivalent to a very large grammar. For example, the rule describing the order of clitic pronouns is equivalent to a set of 64 rules:

CLITICS $\rightarrow$
 CLITICS $\rightarrow$ clr
 CLITICS $\rightarrow$ cld
 ...
 CLITICS $\rightarrow$ clr cld
 CLITICS $\rightarrow$ clr cld cla

However, **Elvex** never builds this set of rules explicitly and remains efficient with large grammars written with alternatives and optional terms.

The use of alternatives and optional terms is therefore recommended to describe the order of words, without regard to the combinatorial problem.

2 Elvex notations

A syntagmatic rule is written

$A \rightarrow B_1 B_2 \dots B_k$ < operational semantics >

Where B_i is written:

- (B_i) if the symbol B_i is optional.
- $C_1|C_2|\dots|C_n$ if B_i is a choice between the C_i

B_i, C_i can be lexical categories or symbols of the grammar.

2.1 Operational Semantics

Notations Let the rule $A \rightarrow B_1 B_2 \dots B_i \dots B_k$

- $\uparrow$: refers to the attribute inherited from A .
- $\downarrow_i$: refers to the attribute inherited from B_i .
- $\Uparrow$: refers to the synthesized attribute of A .
- $\Downarrow_i$: refers to the synthesized attribute of B_i .

The named variables are noted by an identifier preceded by “\$”, the anonymous variables are noted “_”, the constant values are noted by identifiers.

The feature structures are noted $[f_1, f_2, \dots, f_k]$ where f_i is a feature as

- a linked variable feature
- an anonymous variable feature
- **HEAD**:<identifier of a lexeme> a feature for a predicate
- **FORM**: "<form>" a feature for a literal form
- a pair $attr_i : val_i$
 - with $attr_i$ a named attribute
 - and with val_i the value of an attribute that can be:
 - * A named variable
 - * An anonymous variable
 - * A nil value (NIL)
 - * An atomic constant
 - * A set of atomic constants separated by |

- * A literal constant (number)
- * A feature structure
- * A list of feature structures

The lists are noted

- $\langle f_1, f_2, \dots, f_k \rangle$ with f_i as feature structures or lists
- $\langle head :: tail \rangle$ with $head$ as a feature structure or a list and $tail$, a list. $head$ refers to the first item on the list, $tail$ the rest of the list.

If one wishes to bring an operational semantics to the syntagmatic rule, one defines $\langle operational\ semantics \rangle$ which is a list of rules between braces $\{\dots\}$.

A rule may optionally be preceded by rule modifiers such as **@trace**, **@withSpaces**, **@withoutSpaces** or **@unordered**.

- **@trace** traces the application of the rule.
- **@withSpaces** generates spaces between the components of the rule.
- **@withoutSpaces** suppresses spaces between the components of the rule.
- **@unordered** allows the components of the rule to be generated in any order.

Each rule is written:

- $\{\langle \text{list of rules} \rangle\}$ A list of rules.
- **attest** $\langle \text{expr} \rangle$; Explicit guard where $\langle \text{expr} \rangle$ is an boolean expression that must be checked.
- **[f_1, feat_2, \dots, f_k]**; Guard where the feature structure must subsume $\uparrow$. The guard may contain the feature **HEAD** which represents the concept to be generated. Thus, a concept is not necessarily linked to a lexical entry with **Elvex**, unlike usual text generation systems; it can produce a syntactic constraint. Conversely, a syntactic constraint does not necessarily have to be linked to a specific concept.
- **print** $\langle \text{expr} \rangle$; Displays the expression at standard output (useful for debugging).

- `println <expr>;` Does the same and displays a new line.
- `eprint <expr>;` Displays the expression on standard error (useful for debugging without changing generated output).
- `println <expr>;` Does the same on standard error and displays a new line.
- `<expr_1> = <expr_2>;`

An assignment of the expression $\langle expr_1 \rangle$ with the value of the expression $\langle expr_2 \rangle$. The expression $\langle expr_1 \rangle$ is a complex variable, or simple variable, the expression $\langle expr_2 \rangle$ is a constant or a complex variable or a simple variable such that there is an environment³ of the syntagmatic rule to solve it.

The assignments of $\uparrow$ and $\downarrow_i$ are not possible, as these attributes are implicitly computed during text generation and modifying them would result in inconsistencies.

The assignments are limited to these cases:

- Assign a list
 - * $\langle \dots \rangle = \langle \dots \rangle$ Assigns a list to another one.
 - * $\langle \dots \rangle = \X Assigned by the value of the variable $\$X$
- Assign an inherited attribute
 - * $\downarrow i = \$X$ Assigned by the value of the variable $\$X$
 - * $\downarrow i = [\dots]$ Assigned by a constant or variable feature structure.
 - * $\downarrow i = \uparrow$ Assigned by an inherited attribute.
 - * $\downarrow i = \dots \cup \dots$ Assigned by the result of the unification of two expressions (feature structures or attributes).
 - * $\downarrow i = \downarrow j$ Assigned by a synthesized attribute.
 - * $\downarrow i = \uparrow.attr$ or $\downarrow i = \downarrow j.attr$ assigns the value obtained by field access.
- Assign a synthesized attribute
 - * $\uparrow = \$X$

³An environment is an application ϕ from E to F such that E is a set of variables and F a set of constants or variables. The resolution of a variable α in an environment is $\beta = \phi^k(\alpha)$ such that β is a constant. It should be noted that **Elvex** also handles complex variables. In this case $\phi^k(\alpha)$ is replacing variables in terms by their constants.

- * $\uparrow = [\dots]$
- * $\uparrow = \uparrow$
- * $\uparrow = \dots \cup \dots$
- * $\uparrow = \Downarrow j$
- * $\uparrow = \uparrow.\text{attr}$ or $\uparrow = \Downarrow j.\text{attr}$
- Assign a simple variable
 - * $\$X = \Y
 - * $\$X = a$ Assigned by a constant or a literal.
 - * $\$X = \langle \dots \rangle$
 - * $\$X = [\dots]$
 - * $\$X = \uparrow$
 - * $\$X = \dots \cup \dots$
 - * $\$X = \Downarrow j$
 - * $\$X = \uparrow.\text{attr}$ or $\$X = \Downarrow j.\text{attr}$ by field access
 - * $\$X = \langle \text{expr} \rangle$ by evaluating an arithmetical or logical expression
- Subsume a feature structure

A feature structure is subsumed

 - * $[\dots] \subset \uparrow$ by an inherited attribute.
 - * $[\dots] \subset \Downarrow j$ by a synthesized attribute
 - * $[\dots] \subset \$X$ by the value of a variable.
 - * $[\dots] \subset \uparrow.\text{attr}$ or $[\dots] \subset \Downarrow j.\text{attr}$ by field access.
- **if** ($\langle \text{boolExpr} \rangle$) $\langle \text{rules} \rangle$

Interprets operational semantics $\langle \text{rules} \rangle$ if and only if the expression $\langle \text{boolExpr} \rangle$ is verified.
- **if** ($\langle \text{boolExpr} \rangle$) $\langle \text{rules}_1 \rangle$ **else** $\langle \text{rules}_2 \rangle$

Interprets operational semantics $\langle \text{rules}_1 \rangle$ if and only if $\langle \text{boolExpr} \rangle$ is verified, otherwise interpret $\langle \text{rules}_2 \rangle$.
- **deferred** ($\langle \text{boolExpr} \rangle$) $\langle \text{rules} \rangle$

Delays the interpretation of $\langle \text{rules} \rangle$ until $\langle \text{boolExpr} \rangle$ can be evaluated. This is useful when a statement depends on synthesized attributes that are not immediately available.
- **order** $i < j < \dots$;

Adds a linearization constraint between the components of the right-hand side of the current syntagmatic rule. The indexes are

one-based: 1 refers to the first component, 2 to the second one, and so on. For instance, `order 2 < 1;` states that the second component must be generated before the first one. The constraint is projected onto the components that are actually present: if an optional component is absent, the remaining components keep the relative order specified by the chain.

– `order << i;`

Constrains component `i`, if present, to be generated before all other present components of the rule.

– `order >> i;`

Constrains component `i`, if present, to be generated after all other present components of the rule.

– `order by <expr_1, expr_2, ..., expr_k>;`

Sorts the present components of the rule according to the values of the expressions `expr_1`, `expr_2`, ..., `expr_k`. The first expression is the key associated with the first component, the second expression is the key associated with the second component, and so on. Keys must evaluate to numbers. Components are generated in increasing key order; equal keys keep the original right-hand-side order. The expressions can refer to variables or to synthesized features, for example `order by <$R1,$R2,$R3>;` or `order by <↓1.rank, ↓2.rank>;`.

A logical expression `<boolExpr>` is

– `<boolExpr> ∨ <boolExpr>` Or

– `<boolExpr> ∧ <boolExpr>` And

– `¬ <boolExpr>` Negation

– `<boolExpr> ⇒ <boolExpr>` Material implication

– `<boolExpr> ⇔ <boolExpr>` Biconditional

– `<expr> == <expr>` Equality

– `<expr> ≠ <expr>` Difference

– `<expr>` Expression evaluated as Boolean: A feature structure is evaluated as true if it is not NIL or $\perp$ (result of the failure of unification between two feature structures).

The expressions are:

- Integer evaluated as true if not zero
- Double evaluated as true if not zero
- String evaluated as true if not empty
- $\langle \text{expr} \rangle \cup \langle \text{expr} \rangle$ Unification of two feature structures
- $\uparrow$
- $\Uparrow$
- $\downarrow_i$
- $\Downarrow_i$
- $\uparrow.\text{attr}$ and $\Downarrow_i.\text{attr}$ Field access
- $[\dots]$ Feature structure
- NIL Null value of a feature
- $\text{a|b}\dots$ Constant evaluated as true
- $\$X$ Variable
- $\langle \dots \rangle$ List evaluated as true if not null ($\langle \rangle$)

Let's add algebraic expressions and numbers:

- $\langle \text{expr} \rangle < \langle \text{expr} \rangle$
- $\langle \text{expr} \rangle \leq \langle \text{expr} \rangle$
- $\langle \text{expr} \rangle > \langle \text{expr} \rangle$
- $\langle \text{expr} \rangle \geq \langle \text{expr} \rangle$
- $\langle \text{expr} \rangle + \langle \text{expr} \rangle$
- $\langle \text{expr} \rangle - \langle \text{expr} \rangle$
- $\langle \text{expr} \rangle * \langle \text{expr} \rangle$
- $\langle \text{expr} \rangle \% \langle \text{expr} \rangle$
- $\langle \text{expr} \rangle / \langle \text{expr} \rangle$
- $- \langle \text{expr} \rangle$
- Double-precision floating-point format number

Variable evaluation

Variables are evaluated locally at a syntagmatic rule. This defines the scope of a variable.

Here are different methods for solving variables:

- Direct assignment

```
1 S → S {  
2   [vform:NIL];  
3   [tense:$tense, aspect:$aspect, mode:$mode] ⊂ ↑;  
4   if (! $tense)  
5     $tense = present;  
6   ...  
7 }
```

Line 5 indicates that the value of the variable **\$tense** is **present** by default (in the absence of another value previously given). The value of **\$tense** is eventually given by the inherited attribute **↑** on line 3.

Note that the order of evaluation of the rules is indifferent and that it is not necessary to precede rule (3) in the order that they are listed.

- Guard

```
S → NP VP {  
  [subject:$inheritedSubject, $Rest];  
  ...  
}
```

The value of the variable **\$inheritedSubject** is given by the **↑ subject** feature. The value of the variable **\$Rest** is the feature structure that remains when you remove the feature **subject**.

- Subsumption

```
S → NP VP {  
  ...  
  [subject:$synthesizedSubject] ⊂ ↓2;  
  ...  
}
```

The value of the variable **\$synthesizedSubject** is given by the **subject** feature of **↓ 2**.

- Binding a variable within a syntagmatic rule

```

S → NP VP {
  [subject:$inheritedSubject , $Rest];
  [subject:$synthesizedSubject] ⊂ ↓2;
  ↓1 = [subject:$inheritedSubject]
        ∪ [subject:$synthesizedSubject];
  ...
}

```

The variables `$synthesizedSubject` and `$inheritedSubject` are both used within this rule to build the **subject** feature of the inherited attribute $\downarrow 1$.

- Linking a variable within a lexical entry

```

a          v[HEAD: brouillard , subjC:[FORM:" il "],
            objC:[HEAD: brouillard , det:yes ,
            def:no , part:yes , neg:$Neg],
            locCl:[HEAD: _pro , number:sg ,
            person:three] ,
            vform:tensed , vtense:present ,
            mode:indicative ,
            subj:[ person:three , number:sg] ,
            neg:$Neg , fct:none]

```

The variable `$Neg` is linked in this lexical entry to the expression "it is foggy" so that its negative use propagates a negation to the noun:

- (2)
- Il y a du brouillard* (it is foggy)
 - Il n'y a pas de brouillard* (There is no fog)
 - *Il y a de brouillard* ()
 - *Il n'y a pas du brouillard*

2.2 Lexicon

The lexicon is a list of entries separated by semicolons.

Each entry defines the sequence to be produced, followed by a disjunction (noted by the character |) of lexical categories optionally followed by a feature structure.

The sequence to be produced may be:

- A string of characters between "", in which case the sequence will be exactly this one. Note that the sequence may be empty.
- A string of characters between "" containing variables. The sequence produced will be this one with the variables substituted by their respective values.
- FORM: the sequence produced will literally be the one given by the feature FORM; variables in the form are resolved during generation.

The category is an identifier, namely the final symbol of grammar.

The feature structure allows to represent all the constraints imposed by the lexicon.

A few examples:

- "?" interrogativeDot;
- "tag-\$Line" tag[line:\$Line];
- FORM title;
- "elles-mêmes" itself [gender:fm, number:pl, person:three, itself:yes];
- serai aux_être [aux:être, voice:active, finite:yes, mode:indicative, vtense:future_anterieur, vform:tensed, subj:[person:one, number:sg]];
- // Lexical input "temperature" producing
 //"La température est de xxx degrés" (the temperature is xxx degrees Celsius"
 est v[HEAD:température,
 subjC:[HEAD:température, number:sg, det:yes, def:yes],
 pobjC:[HEAD:degré, number:pl, pcas:de, det:yes,
 detNum:[HEAD:num, value:\$Deg]],
 vform:tensed, vtense:present, mode:indicative,
 subj:[person:three, number:sg], fct:none, value:\$Deg]

 //"la température est douce" (the temperature is mild)
 |v[HEAD:température, subjC:[HEAD:température, number:sg,

```

det:yes, def:yes], modC:[HEAD:doux, number:sg, gender:fm],
vform:tensed, vtense:present, mode:indicative,
subj:[person:three, number:sg], fct:few]

// "la température est élevée" (the temperature is high)
|v[HEAD:température, subjC:[HEAD:température, number:sg,
det:yes, def:yes], modC:[HEAD:élevé, number:sg, gender:fm],
vform:tensed, vtense:present, mode:indicative,
subj:[person:three, number:sg], fct:much];

• // Lexical entry of "temperature".
// producing "Il fait chaud" (It's hot)
fait v[HEAD:température, subjC:[FORM:"il"],
modC:[HEAD:chaud, number:sg, gender:ms], vform:tensed,
vtense:present, mode:indicative,
subj:[person:three, number:sg], neg:$Neg, fct:high]

// "Il fait très chaud" (It's very hot)
|v[HEAD:température, subjC:[FORM:"il"], modC:[HEAD:chaud,
number:sg, gender:ms, mod:<[HEAD:très]>], vform:tensed,
vtense:present, mode:indicative, subj:[person:three,
number:sg], neg:$Neg, fct:very_high]

// "Il fait très froid" (It's very cold)
|v[HEAD:température, subjC:[FORM:"il"], modC:[HEAD:froid,
number:sg, gender:ms, mod:<[HEAD:très]>], vform:tensed,
vtense:present, mode:indicative, subj:[person:three,
number:sg], neg:$Neg, fct:very_low]

// "Il fait froid" (It's cold)
|v[HEAD:température, subjC:[FORM:"il"], modC:[HEAD:froid,
number:sg, gender:ms], vform:tensed, vtense:present,
mode:indicative, subj:[person:three, number:sg],
neg:$Neg, fct:low];

• // Lexical entry of "rain" producing
// "Il y a une pluie fine" (A fine rain fell)
a v[HEAD:pleuvoir, subjC:[FORM:"il"],
objC:[HEAD:pluie, det:yes, number:sg, qual:yes,

```



```

    def:no, mod:<[HEAD:fin, pos:post]>],
    locCl:[HEAD:_pro, number:sg, person:three],
    vform:tensed, vtense:present, mode:indicative,
    subj:[person:three, number:sg], neg:$Neg, fct:few]

// "il y a un peu de pluie" (It is raining a bit)
|v[HEAD:pleuvoir, subjC:[FORM:"il"],
  objC:[HEAD:pluie, det:yes, number:sg,
    detForm:[FORM:"un peu de"]],
  locCl:[HEAD:_pro, number:sg, person:three],
  vform:tensed, vtense:present, mode:indicative,
  subj:[person:three, number:sg], neg:$Neg, fct:not_much]

// "il y a beaucoup de pluie" (There is plenty of rainfall)
|v[HEAD:pleuvoir, subjC:[FORM:"il"],
  objC:[HEAD:pluie, det:yes, number:sg,
    detForm:[FORM:"beaucoup de"]],
  locCl:[HEAD:_pro, number:sg, person:three],
  vform:tensed, vtense:present, mode:indicative,
  subj:[person:three, number:sg], neg:$Neg, fct:much]

// "il y a de fortes précipitations" (It is raining so much)
|v[HEAD:pleuvoir, subjC:[FORM:"il"],
  objC:[HEAD:précipitation,
    det:yes, number:pl, qual:yes, def:no, mod:<[HEAD:fort,
      pos:pre]>], locCl:[HEAD:_pro, number:sg, person:three],
    vform:tensed, vtense:present, mode:indicative,
    subj:[person:three, number:sg], neg:$Neg, fct:a_lot];

```

2.3 Input

The input is summarized by the type of phrase or text you want to generate, followed by a feature structure containing the concept and the illocutionary structure to be expressed.

HEAD allows you to define the named concept to be generated.

examples

- sentence [HEAD:raining, neg:yes]

Simple entry for the concept of *raining* producing

- (3) a. *Il ne pleut pas* (It is not raining)

b. *Il n'y a pas de pluie* (There is no rain)

- sentence
[HEAD:to_ask,
i:[HEAD:menuisier, id:3, number:sg, gender:ms, def:yes],
iii:[HEAD:apprenti, id:6, def:yes, gender:ms, number:sg],
ii:[HEAD:scier,
i:[idref:6],
ii:[HEAD:poutre, number:sg, def:yes]
]
]

Complex concept where the features `id` and `idref` make it possible to constrain co-references.

This entry produces

- (4) *Le menuisier demande à l'apprenti de scier la poutre*
The carpenterasks the apprentice to saw the beam
The carpenter asks the apprentice to saw the beam

- sentence
[HEAD:raining,
tense:future,
modSType:time,
modS:<[FORM:"lundi 18 mars 2019", type:time]>
]

Concept producing an adverbial on the basis of the constant "lundi 18 mars 2019" (Monday, the 18th of March 2019). This entry produces:

- (5) *Il va pleuvoir le lundi 18 mars 2019.* (It will rain on Monday, March 18, 2019)

- sentence
[HEAD:cause,
i:[HEAD:raining]]
ii:[HEAD:se_couvrir, i:[HEAD:you], modality:devoir]
]

Input for the following generated sentences:

- (6) a. *Il pleut, tu dois te couvrir* (It is raining, you have to dress warmly)

- b. *Il pleut, par conséquent tu dois te couvrir* (It is raining, therefore you have to dress warmly)
- c. *Tu dois te couvrir s'il pleut* (You have to dress warmly if it is raining)
- d. *Tu dois te couvrir car il pleut* (You have to dress warmly because it is raining)
- e. *S'il pleuvait, tu devrais te couvrir* (If it rained, you should dress warmly)
- f. *S'il pleut, tu devras te couvrir* (If it is raining, you should dress warmly)
- g. *Il pleuvrait, tu devrais te couvrir* (It would rain, you should dress warmly)

- sentence

```
[HEAD:initiative,
 i:[HEAD:Jean],
 objC:[lexFct:Magn, mod:<[HEAD:audacieux]>],
 lexFct:Oper1
]
```

Input containing features to fix lexical functions. The production is

- (7) *Jean prend une initiative très audacieuse.* (Jean takes a very bold initiative)

- sentence

```
[HEAD:initiative,
 i:[HEAD:Jean],
 objC:[lexFct:Magn],
 lexFct:Oper1
]
```

Input for

- (8) *Jean prend une belle initiative.* (Jean takes a fine initiative)

- sentence

```
[HEAD:to_break,
 i:[HEAD:Jean],
 ii:[HEAD:carafe, number:sg, def:yes,
 mod:<[HEAD:little],[HEAD:white]>],
```

```

    diathesis:passive, tense:past, neg:yes,
    modV:<[HEAD:roughly]>
]

```

Input containing a negative passive feature. The production is

- (9) *La petite carafe blanche n'avait pas été cassée brutalement par Jean.* (The small white carafe had not been broken brutally by Jean.)

3 Design pattern (to be completed)

A design pattern applies in a general case which can be summarized by some constraints. It is not associated with a specific event, but to a recurring organization of the grammar for which a single and coherent answer must be provided.

A - Local propagation of synthesized attributes

Context: A feature A that must constrain a result R, where R depends only on the lexicon or grammar.

Examples of applications:

- Production of a constrained noun in an idiomatic phrase (*breaking the ice*)
- Conjugation to the passive voice

The general rule is written as follows

```

1 X → Y {
2   [A:$v, $rest];
3   [AResultion:R] ⊂ ↓1;
4   ↓1 = [R:$v, $rest];
5 }

```

Example 1: The sentence *We basically got taken to the cleaners on that deal.* introduces the fixed noun *cleaners* which comes from the verb *take one to the cleaner*. Here the agent of the passive sentence is not represented.

```

1 VN → V (PP) PP {
2   [obj:NIL, obl:$Obl];
3   [subj:$subj] ⊂ ↑;

```

```

4 | ↓1 = ↑;
5 | ↓3 = $Obl;
6 | [oblC:$oblSynt] ⊂ ↓1;
7 | if (#2) {
8 |     attest $oblSynt;
9 |     ↓2 = $oblSynt;
10| }
11| else {
12|     attest ¬$oblSynt;
13| }
14| }
15|
16| taken verb[HEAD:take_one_to_the_cleaners ,
17|     constI:[HEAD:cleaner , number:pl , def:yes]];

```

Example 2: The agent of a clause is carried out by the grammatical function subject for conjugation to the active voice, and by the oblique function in *by* for a passive voice conjugation.

```

1 | // agent as subject
2 | Sentence → Sentence {
3 |     [agent:$agent , $rest];
4 |     [agentRealization:subject] ⊂ ↓1;
5 |     ↓1 = [subject:$agent , $rest];
6 |     ↑ = ↓1;
7 | }
8 |
9 | // agent as oblique
10| Sentence → Sentence {
11|     [agent:$agent , $rest];
12|     [agentRealization:oblBy] ⊂ ↓1;
13|     ↓1 = [byObj:$agent , $rest];
14|     ↑ = ↓1;
15| }
16|
17| // patient as object
18| Sentence → Sentence {
19|     [patient:$patient , agent:NIL , $rest];
20|     [patientRealization:object] ⊂ ↓1;
21|     ↓1 = [object:$patient , $rest];
22|     ↑ = ↓1;

```

```

23 }
24
25 // patient as subject
26 Sentence → Sentence {
27   [patient:$patient , agent:NIL , $rest ];
28   [patientRealization:subject] ⊂ ↓1;
29   ↓1 = [subject:$patient , $rest ];
30   ↑ = ↓1;
31 }
32
33 // passive voice
34 Sentence → NP VERB PP {
35   [subject:$subject , parObl:$parObl , $rest ];
36   ↓1 = $subject;
37   ↓2 = $rest ∪ [voice:passive];
38   ↓3 = $parObl;
39   ↑ = [agentRealization:oblBy ,
40         patientRealization:subject ];
41 }
42
43 // active voice
44 Sentence → NP VERB NP {
45   [subject:$subject , object:$object , $rest ];
46   ↓1 = $subject;
47   ↓2 = $rest ∪ [voice:active];
48   ↓3 = $object;
49   ↑ = [agentRealization:subject ,
50         patientRealization:object ];
51 }

```

B - Transformation of the attribute inherited by a syntactic head

Context: An inherited attribute A that must constrain a realization R by a new predicate in generation.

Examples of applications:

- Use of a verbal tense auxiliary or a modal auxiliary
- Compound conjugation in French

The general rule is written as follows

```

1 X → Y {
2   [A:K];
3   [HEAD:$HEAD, A:k, $Rest] ⊂ ↑;
4   ↓1 = [HEAD:L, arg:[HEAD:$HEAD, $Rest]];
5 }

```

Example: In French, the aspect is reflected in conjugated forms, but also in compound verb constructions (*être en train de* + infinitive, to be in the process of doing sth).

```

1 sentence → sentence {
2   [aspect:progressive];
3   [HEAD:$HEAD, aspect:progressive, i:$I, $Rest] ⊂ ↑;
4   ↓1 = [HEAD:être_en_train_de,
5         i:[HEAD:$HEAD, i:$I, $Rest]];
6   ↑ = ↓1;
7 }

```

4 The elvex command

Elvex is a Natural Language Generator. It takes as input a concept, a local lexicon, a compacted lexicon, and a grammar, and then outputs a text corresponding to the concept written in natural language.

Compilation and full installation of Elvex

The full compilation of **Elvex** requires the following programs on your machine:

- g++
- flex
- bison
- cmake
- libxml2-dev

The full command is:

```
cmake .  
make  
sudo make install
```

4.1 Trying Elvex

```
. ./try-me.sh
```

4.2 Elvex command

The `elvex` command is:

```
elvex [options] [<input>]*
```

The options are:

`-h|--help`

Displays help and exits

`-v|--version`

Displays the version number and exits

`-V|--verbose`

Enables verbose mode.

`-a|--reduce-all`

Reduces all rules during the generation process. This mode allows fine debugging and requires an understanding of the generation process to be mastered.

`-t|--trace`

Displays the reduced rules during the generation process. This mode is useful for grammar debugging.

`-r|--random`

Displays only one randomly selected output.

`--seed <number>`

Sets the random seed for reproducible random generation.

`-f|--first`
Displays only the first output.

`--strategy <exhaustive|sample|beam>`
Selects the derivation strategy. The default strategy is **exhaustive**.

`--max-rule-choices <integer>`
Defines the maximum number of rule or disjunction choices opened in **sample** mode.

`--beam-width <integer>`
Defines the maximum number of items kept per state in **beam** mode.

`--macros-file <file>`
File containing macro definitions.

`--rules-file <file>`
File containing the rules.

`--lexicon-file <file>`
File containing the local lexicon.

`--input-file <file>`
File containing the input.

`--compact-ed-lexicon-file <path/file> or --clf <path/file>`
Prefix of the compacted lexicon files. The files `<path/file>.tbl` and `<path/file>.fsa` are used.

`--max-length <integer>`
Defines the maximum length of the sequence to be produced.

`--max-usages <integer>`
Defines the maximum number of times the same rule can be used.

`--max-items <integer>`
Defines the maximum cardinality of the item sets.

`--max-time <integer>`
Defines the number of seconds before the program stops.

`--max-attempts <integer>`

Defines the maximum number of attempts for random generation.

4.3 Grammar debugging

Grammar debugging is performed with the `elvex` command using trace options. For example:

```
elvex --trace \  
  --rules-file rules.elvex \  
  --lexicon-file lexicon.elvex \  
  <input>
```

With tracing enabled, `elvex` produces an HTML document on the standard output. You must use a browser to view this document properly and review the computation.

Where the specific options are:

`--trace-all`

Trace all generation steps

`--trace-init`

Trace the initial items.

`--trace-stage`

Trace the item sets for each generation stage.

`--trace-close`

Trace the item set closure.

`--trace-shift`

Trace the items for each lexical entry.

`--trace-reduce`

Trace the items for each reduction.

`--trace-action`

Trace the operational semantics results.

4.4 elvexlexicon for managing compacted lexicons

The `elvexlexicon` command builds and inspects compacted lexicons. Compacted lexicons are intended for open lexical categories such as nouns, verbs, adjectives and adverbs. Closed or grammatical categories, such as prepositions, determiners, pronouns or complementizers, may remain in the local lexicon loaded directly by `elvex` with `--lexicon-file`.

The command is:

```
elvexlexicon [global options] <build|consult|list>
```

The action is mandatory:

build

Builds the compacted lexicon from a pattern file and a morphological file.

list

Lists the content of an existing compacted lexicon.

consult

Consults an existing compacted lexicon interactively.

The global options are:

-h|--help

Displays help and exits.

-v|--version

Displays the version number and exits.

-V|--verbose

Enables verbose mode.

--compacted-lexicon-file <path/file> or --clf <path/file>

Prefix of the compacted lexicon files. The files `<path/file>.tbl` and `<path/file>.fsa` are used.

--macros-file <file>

File containing macro definitions used in the pattern and morphological files.

`--pattern-file <file>`

Pattern file describing lexical entries at the lexeme level. It consists of lines of the form:

```
lexeme  pos  lemma  features
```

The **lexeme** is the lexical unit selected by the generation model. It may correspond to a simple word, a collocation or an idiomatic construction. The **lemma** identifies the morphological paradigm with which the lexeme is associated.

`--morpho-file <file>`

Morphological file describing inflected forms at the lemma level. It consists of lines of the form:

```
form  pos  lemma  features
```

The **form** is the produced surface form. The **lemma** is used to match a morphological entry with one or more pattern entries.

The compacted lexicon is built by combining pattern entries and morphological entries that share the same **pos** and **lemma**. Their feature structures are unified. This keeps a clear distinction between the lexeme, which represents a lexical choice in the generation model, and the lemma, which represents a morphological paradigm.

For example, a pattern file may contain adjective classes and their linearization rank:

```
LARGE  adj  large  [HEAD:large , type:size , rank:1]
PINK   adj  pink   [HEAD:pink , type:color , rank:4]
IRON   adj  iron   [HEAD:iron , type:material , rank:6]
```

and the morphological file may contain the corresponding surface forms:

```
large  adj  large  []
pink   adj  pink   []
iron   adj  iron   []
```

After compilation, the resulting entries can be used by grammar rules. For instance, the synthesized feature **rank** can later be exploited by a linearization instruction such as:

```
order by <↓1.rank , ↓2.rank , ↓3.rank>;
```

Command to build a compacted lexicon

```
elvexlexicon --clf <path/file> \  
  --macros-file <macros-file> \  
  --pattern-file <pattern-file> \  
  --morpho-file <morpho-file> \  
  build
```

The `--macros-file` option is needed only if the pattern or morphological files use macros.

Command to list a compacted lexicon

```
elvexlexicon --clf <path/file> list
```

Command to consult a compacted lexicon

```
elvexlexicon --clf <path/file> consult
```

The compacted lexicon is a set of lexical entries of the type

```
pos#lexeme form#FS|form#FS...
```

- `pos` is a part of speech
- `lexeme` is the lexeme given by the `HEAD` feature
- `form` is the produced form, represented as a string of UTF-8 characters
- `FS` is a feature structure